

Introduction to Machine Learning

Core concepts: learning paradigms, the modeling workflow, overfitting, and optimization

1. What Is Machine Learning?

Machine learning is a subfield of artificial intelligence concerned with building systems that improve their performance on a task through experience. Rather than a programmer writing explicit rules for every situation, a learning algorithm examines data, discovers patterns, and encodes those patterns in a model that can make predictions on new, unseen inputs.

A classic illustration is email spam filtering. Writing hand-coded rules to catch every possible spam variant is hopeless: spammers constantly change tactics. A machine learning approach instead feeds the algorithm thousands of emails already labeled as spam or not spam. The algorithm learns which characteristics — certain words, suspicious links, odd formatting — tend to accompany spam, and applies that knowledge to incoming mail.

The core vocabulary

- **Example / instance:** a single data point, such as one email or one house listing.
- **Features:** the measurable properties of an example, for instance word frequencies or square footage.
- **Label / target:** the quantity we want to predict, such as spam-or-not or sale price.
- **Model:** the learned mapping from features to predictions.
- **Training:** the process of fitting a model to data.

2. Types of Machine Learning

Machine learning tasks are usually grouped into three broad families, distinguished by the kind of feedback the algorithm receives.

Supervised learning

In supervised learning, every training example comes with the correct answer — the label. The algorithm's job is to learn the mapping from features to labels. Predicting house prices from property characteristics is *regression* (the label is continuous); classifying emails as spam or not spam is *classification* (the label is categorical). Most commercial machine learning today is supervised learning.

Unsupervised learning

In unsupervised learning there are no labels at all. The algorithm must find structure in the data on its own. *Clustering* groups similar customers together for marketing; *dimensionality reduction* compresses high-dimensional data while preserving its essential shape, which is useful for visualization and for speeding up later supervised steps.

Reinforcement learning

In reinforcement learning an *agent* takes actions in an environment and receives rewards or penalties. Through trial and error it learns a *policy* — which action to take in each situation — that maximizes long-term reward. Game-playing systems and some robotics controllers are trained this way.

3. The Learning Workflow

A typical supervised project follows a repeatable workflow. First, data is collected and cleaned: missing values are handled, outliers are examined, and features are engineered into a numerical form algorithms can consume. Second, the data is split into training, validation, and test portions, a practice whose importance cannot be overstated and which is examined in detail in Section 4. Third, a model family is chosen and trained — the algorithm adjusts its internal parameters to reduce error on the training data. Fourth, the model is evaluated on data it has never seen, to estimate how it will behave in the real world. Finally, the validated model is deployed and monitored, because real-world data drifts over time.

Two failure modes haunt every stage of this workflow: overfitting and underfitting. They are so central that they deserve their own section.

4. Training, Validation, and Test Splits

A model that is evaluated on the same data it trained on will look deceptively good. To obtain an honest estimate of future performance, the available labeled data is divided into three disjoint subsets.

- **Training set** (often ~70%): the only data the learning algorithm may fit to. Parameters are adjusted here.
- **Validation set** (often ~15%): used to compare model variants and tune hyperparameters such as regularization strength or tree depth.
- **Test set** (often ~15%): locked away until the very end. It is examined exactly once, to report final performance.

The cardinal rule is that the test set must never influence any modeling decision — not feature selection, not hyperparameter tuning, not early stopping. Peeking at the test set, even indirectly, leaks information and produces optimistic estimates that collapse on genuinely new data. When data is scarce, *cross-validation* rotates the validation role across folds so every example contributes to both training and validation; the companion document *Machine Learning Evaluation* covers this technique in depth.

5. Overfitting and Underfitting

Overfitting occurs when a model learns the training data too well — including its noise and quirks — and therefore generalizes poorly to new examples. A massively deep decision tree, for example, can memorize every training instance and still fail on the next customer who walks in. The signature of overfitting is a large gap between training performance (excellent)

and validation performance (mediocre).

Underfitting is the opposite problem: the model is too simple to capture the underlying pattern, so it performs poorly even on the training data. Fitting a straight line to data that curves is the textbook case.

Practitioners fight overfitting with several tools: gathering more training data, *regularization* (penalizing overly complex models), simplifying the model family, and *early stopping* (halting training when validation error starts rising). The definitive way to *detect* overfitting is to plot learning curves — training versus validation error as a function of training-set size or training iterations — and watch for the characteristic divergence. The companion document *Machine Learning Evaluation* walks through learning curves step by step.

6. The Bias-Variance Tradeoff

Overfitting and underfitting are two faces of the *bias-variance tradeoff*, one of the most important ideas in machine learning. *Bias* is error from mistaken assumptions — a model with high bias systematically misses the true pattern (underfitting). *Variance* is error from sensitivity to the particular training sample — a model with high variance swings wildly when the data changes slightly (overfitting).

Total expected error decomposes, roughly, into bias squared plus variance plus irreducible noise. Simple models have high bias and low variance; complex models have low bias and high variance. Good modeling is the search for the sweet spot in between, and techniques like regularization and cross-validation exist precisely to help find it.

7. Gradient Descent: The Optimization Workhorse

Almost every modern model is trained by minimizing a *loss function* — a number that measures how wrong the model's predictions are. *Gradient descent* is the algorithm that performs this minimization. It starts from an initial guess for the model parameters, computes the gradient (the direction of steepest increase) of the loss, and takes a small step in the opposite direction. Repeating this process walks downhill until the loss can no longer be reduced much further.

The *learning rate* controls the step size: too large, and the algorithm overshoots and diverges; too small, and training crawls. Variants such as *stochastic* gradient descent (updating on one example at a time) and *mini-batch* gradient descent (updating on small random subsets) add noise that helps escape shallow local minima and dramatically speeds up training on large datasets.

Gradient descent is not specific to any one model family — it trains linear models, neural networks, and many others. The companion document *Neural Networks and Deep Learning* shows how the backpropagation algorithm is, at its heart, an efficient way of computing the gradients that gradient descent needs.

8. Evaluating Models: A First Look

The simplest evaluation metric for classification is *accuracy*: the fraction of predictions the model gets right. It is intuitive but dangerously misleading on imbalanced problems — a model that always predicts the majority class can score 99% accuracy on a dataset with 1% positives while being utterly useless.

That is why practitioners reach for richer metrics: *precision* (of the items flagged positive, how many truly are?), *recall* (of the truly positive items, how many did we catch?), their harmonic mean the *F1 score*, and ranking metrics like *ROC-AUC*. Choosing among them depends on the cost of each kind of mistake, a topic treated thoroughly in the companion document *Machine Learning Evaluation*.